🛠️ Setup & Troubleshooting — MLX Python

> Part of the MLX Python Documentation. Function details are in the MLX Reference.

This guide covers **mlx_CLXV v2.2** (the Python MLX given in 42 projects), on Linux. Everything here was tested by building the library from source on Ubuntu 24.04.

---

1. What You Need

MLX Python is a C library (`libmlx.so`) wrapped by a small Python class. To build and run it you need:

Need	Why	Ubuntu / Debian package
C compiler	the Makefile uses `clang`	`clang`
XCB	opens the window, reads events	`libxcb1-dev`, `libxcb-keysyms1-dev`
Vulkan	draws on the GPU	`libvulkan-dev`
A Vulkan driver	actually runs the drawing	`mesa-vulkan-drivers`
zlib, libbsd	PNG loading, utilities	`zlib1g-dev`, `libbsd-dev`
Python 3 + venv	runs your project	`python3`, `python3-venv`

On your own machine:

```
sudo apt install clang libxcb1-dev libxcb-keysyms1-dev libvulkan-dev \
                 mesa-vulkan-drivers zlib1g-dev libbsd-dev python3-venv
```

> On the 42 campus machines these are normally already installed — skip to step 2.

> This MLX version is **Linux only** (XCB + Vulkan). It does not run natively on macOS.

---

2. Build the Library

Inside the `mlx_CLXV` folder you received:

```
cd mlx_CLXV
make
```

`make` does three things:

1. runs `configure.sh`, which checks every dependency and tells you exactly which one is missing
2. compiles `libmlx.so`
3. packages it into a Python wheel: `mlx-2.2-py3-none-any.whl`

If the build succeeds, these two files appear in the folder:

```
libmlx.so
mlx-2.2-py3-none-any.whl
```

> You may see `./pybuild.sh: deactivate: not found` at the end. It is harmless: the wheel was still built. Check that the `.whl` file exists.

---

3. Install It in a Virtual Environment

```
python3 -m venv venv
source venv/bin/activate
pip install mlx_CLXV/mlx-2.2-py3-none-any.whl
```

> ⚠️ **Never run `pip install mlx`.** There is an unrelated package with the same name on PyPI (Apple's machine-learning framework). Always install the **`.whl` file** from your `mlx_CLXV` folder.

The wheel contains the compiled `libmlx.so`, so you don't need to set `LD_LIBRARY_PATH`.

---

4. Check That It Works

```
from mlx import Mlx

mlx = Mlx()
mlx_ptr = mlx.mlx_init()
print("mlx_ptr:", mlx_ptr)
```

If it prints a number, MLX is working. If it prints `None` or crashes, see Troubleshooting.

> The import is `from mlx import Mlx` (lowercase package). The official README shows `from Mlx import Mlx`, which fails with `ModuleNotFoundError`.

Then try the minimum working program from the reference.

---

5. Using a Makefile in Your Project

A common pattern (used in A-Maze-ing) is a Makefile with an `install` rule that creates the venv and installs the wheel, building MLX first if the wheel doesn't exist yet:

```
VENV = venv
PIP  = $(VENV)/bin/python3 -m pip

install:
    python3 -m venv $(VENV)
    @if [ ! -f mlx_CLXV/mlx-2.2-py3-none-any.whl ]; then $(MAKE) -C mlx_CLXV; fi
    $(PIP) install mlx_CLXV/mlx-2.2-py3-none-any.whl

run:
    $(VENV)/bin/python3 main.py
```

> Don't commit a `.whl` or `libmlx.so` built on your machine and expect it to work on another one. If the other machine is missing a library, it fails. Rebuilding with `make` is safer.

---

6. Running on WSL (Windows)

MLX needs a graphical display (X server) and a Vulkan driver.

- **Windows 11 + WSL2:** WSLg provides the display automatically. Check with `echo $DISPLAY` — it should print something like `:0`.
- Install `mesa-vulkan-drivers` (step 1). It includes a software Vulkan driver, so MLX works even without GPU access, just slower.
- To check Vulkan, install `vulkan-tools` and run `vulkaninfo --summary`. It should list at least one device.
- **Windows 10** has no WSLg: you need an external X server (e.g. VcXsrv) and to set `DISPLAY` yourself.

---

7. Troubleshooting

Symptom	Cause	Fix
`make` → `configure.sh` says something is `not found`	missing dependency	install the package from the table in step 1
`make` → `clang: not found`	no clang	`sudo apt install clang`
`ModuleNotFoundError: No module named 'mlx'`	wheel not installed, or venv not active	`source venv/bin/activate`, then reinstall the wheel
`ModuleNotFoundError: No module named 'Mlx'`	wrong import	use `from mlx import Mlx`
Strange `mlx` errors about arrays, Metal, etc.	Apple's `mlx` was installed instead	`pip uninstall mlx`, then install the `.whl`
Segmentation fault right at `mlx_init()`	no display available (`DISPLAY` empty)	run from a graphical session; on WSL see step 6; over SSH use `ssh -X`
`Mlx - Vk: ... Missing required Vulkan extensions` + `Mlx init: Can't initialize gpu`, `mlx_init()` returns `None`	no Vulkan driver	`sudo apt install mesa-vulkan-drivers`; check with `vulkaninfo --summary`
`mlx_xpm_file_to_image` returns `(None, 0, 0)`	wrong path, or invalid XPM	paths are relative to **where you run Python**, not to the script; check the file opens
PNG doesn't load / looks wrong	the PNG loader is incomplete	convert to XPM: `magick input.png output.xpm`
`Ctrl+C` doesn't stop the program	the signal is raised inside an MLX callback and ignored	close with ESC / the window button (see the reference), or `Ctrl+\`
`Exception ignored on calling ctypes callback function`	your hook raised an exception	see below

Exceptions inside hooks

When a hook (key, mouse, loop…) raises an exception, Python prints the traceback starting with `Exception ignored on calling ctypes callback function` and **the loop keeps running**. The program doesn't stop, but your state may be half updated — the rest of that callback was skipped.

Always read the terminal when something on screen looks wrong: the real error is often printed there while the window keeps going.

The loop hook runs very often

The loop hook is called continuously — thousands of times per second when the hooks are cheap. So:

- use a **dirty flag** and only redraw when something changed (see the reference)
- for animations, use time (`time.monotonic()`) rather than counting frames, otherwise the speed changes from machine to machine

---

8. Useful Commands

```
# Check dependencies without building
./configure.sh

# Rebuild everything from scratch
make re

# See what libmlx.so needs (look for "not found")
ldd venv/lib/python3*/site-packages/mlx/libmlx.so

# Read the official man pages
man ./man/man3/mlx.3
```